ARCHON — Episode 2: Dot Products and Projections

March 24, 2026 • Phase: 1 • linear_algebra

Episode 2: Dot Products and Projections

Yesterday's Recap

Welcome back! Yesterday, we laid the foundation of linear algebra by exploring **vectors and spaces**. Let's quickly review the key concepts you learned:

- **Vectors** are ordered lists of numbers that represent points or directions in space
- We can visualize vectors as arrows from the origin to a point
- **Vector operations** include addition (tip-to-tail method) and scalar multiplication (stretching/shrinking)
- **Vector spaces** are collections of vectors closed under addition and scalar multiplication
- The **standard basis** in 2D consists of $\mathbf{i} = [1, 0]$ and $\mathbf{j} = [0, 1]$
- Any vector can be written as a **linear combination** of basis vectors: $\mathbf{v} = a\mathbf{i} + b\mathbf{j}$
- **Magnitude** (or norm) of a vector $\mathbf{v} = [v_1, v_2]$ is $||\mathbf{v}|| = \sqrt{v_1^2 + v_2^2}$

Today, we'll build on these concepts to understand how vectors relate to each other through **dot products** and how we can decompose vectors using **projections**. These tools are fundamental to machine learning — they help us measure similarity, reduce dimensionality, and understand the geometry of high-dimensional data.

Today's Topic: Dot Products and Projections

Imagine you're building a recommendation system for movies. You represent each movie as a vector where each dimension corresponds to a feature (action level, romance level, comedy level, etc.). How do you measure if two movies are similar? How do you find the "action component" of a romantic comedy? These are exactly the questions that dot products and projections help us answer.

The **dot product** is perhaps the single most important operation in machine learning. It appears everywhere: - In neural networks, every neuron computes a dot product between weights and inputs - In similarity measures, cosine similarity is based on dot products - In optimization, gradients point in directions determined by dot products - In dimensionality reduction (like PCA), we project data using dot products

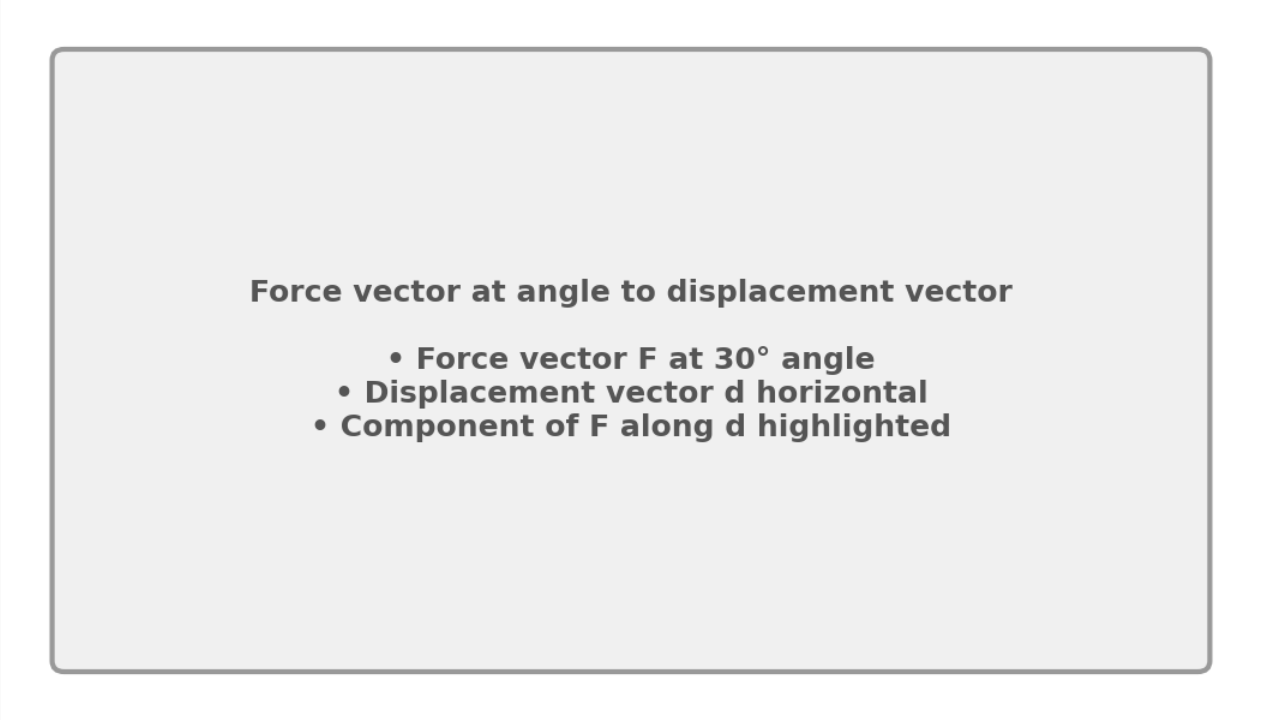
By the end of today's episode, you'll understand: 1. What the dot product means geometrically and algebraically 2. How to use dot products to measure angles and similarity between vectors 3. How to project one vector onto another 4. Why orthogonality (perpendicularity) is such a powerful concept 5. How these concepts apply directly to machine learning algorithms

Let's dive in with an intuitive example before we formalize anything.

Core Concepts

Intuition: What is a Dot Product?

Let's start with a physical analogy. Imagine you're pushing a heavy box across the floor:



The work you do depends on two things: 1. How hard you push (magnitude of force) 2. How much of your push is in the direction of motion

If you push straight down, the box doesn't move horizontally — your force doesn't contribute to horizontal displacement. If you push horizontally, all your force contributes to motion. The dot product captures exactly this idea: **how much of one vector points in the direction of another**.

Geometric Definition of Dot Product

The dot product of two vectors $\mathbf{a}$ and $\mathbf{b}$ is defined geometrically as:

$$\mathbf{a} \cdot \mathbf{b} = ||\mathbf{a}|| \cdot ||\mathbf{b}|| \cdot \cos(\theta)$$

Where: - $||\mathbf{a}||$ is the magnitude (length) of vector $\mathbf{a}$ - $||\mathbf{b}||$ is the magnitude (length) of vector $\mathbf{b}$ - θ is the angle between the two vectors - $\cos(\theta)$ is the cosine of that angle

This formula tells us that the dot product depends on: 1. The lengths of both vectors 2. How aligned they are (the angle between them)

Let's visualize this with code:

```
import numpy as np
import matplotlib.pyplot as plt

# Define two vectors
a = np.array([3, 2])  # First vector
b = np.array([2, 1])  # Second vector

# Calculate dot product geometrically
magnitude_a = np.linalg.norm(a)  # Length of vector a: sqrt(3^2 + 2^2) = sqrt(13)
magnitude_b = np.linalg.norm(b)  # Length of vector b: sqrt(2^2 + 1^2) = sqrt(5)

# Calculate angle between vectors using inverse cosine
# We'll derive the algebraic formula for dot product soon!
dot_product = np.dot(a, b)  # For now, use NumPy's function
cos_theta = dot_product / (magnitude_a * magnitude_b)  # Rearrange the geometric formula
theta = np.arccos(cos_theta)  # Get angle in radians

print(f"Vector a: {a}")
print(f"Vector b: {b}")
print(f"Magnitude of a: {magnitude_a:.3f}")
print(f"Magnitude of b: {magnitude_b:.3f}")
print(f"Angle between vectors: {np.degrees(theta):.1f} degrees")
print(f"Dot product: {dot_product}")

# Visualize the vectors and angle
plt.figure(figsize=(8, 8))
plt.quiver(0, 0, a[0], a[1], angles='xy', scale_units='xy', scale=1, color='blue', width=0.006)
plt.quiver(0, 0, b[0], b[1], angles='xy', scale_units='xy', scale=1, color='red', width=0.006)

# Add labels
plt.text(a[0]/2, a[1]/2 + 0.2, 'a', fontsize=12, color='blue')
plt.text(b[0]/2 - 0.2, b[1]/2, 'b', fontsize=12, color='red')

# Draw angle arc
angle_arc = np.linspace(0, theta, 100)
arc_radius = 0.8
plt.plot(arc_radius * np.cos(angle_arc), arc_radius * np.sin(angle_arc), 'g--', linewidth=2)
plt.text(0.5, 0.2, f'θ = {np.degrees(theta):.1f}°', fontsize=10, color='green')

plt.xlim(-1, 4)
plt.ylim(-1, 3)
plt.grid(True, alpha=0.3)
plt.axhline(y=0, color='k', linewidth=0.5)
plt.axvline(x=0, color='k', linewidth=0.5)
plt.xlabel('x')
plt.ylabel('y')
plt.title('Geometric View of Dot Product')
```

```
plt.axis('equal')
plt.show()
```

Algebraic Definition of Dot Product

While the geometric definition gives us intuition, the **algebraic definition** is what we actually use for computation. For two vectors in n dimensions:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{n} a_i b_i$$

In 2D, if $\mathbf{a} = [a_1, a_2]$ and $\mathbf{b} = [b_1, b_2]$:

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2$$

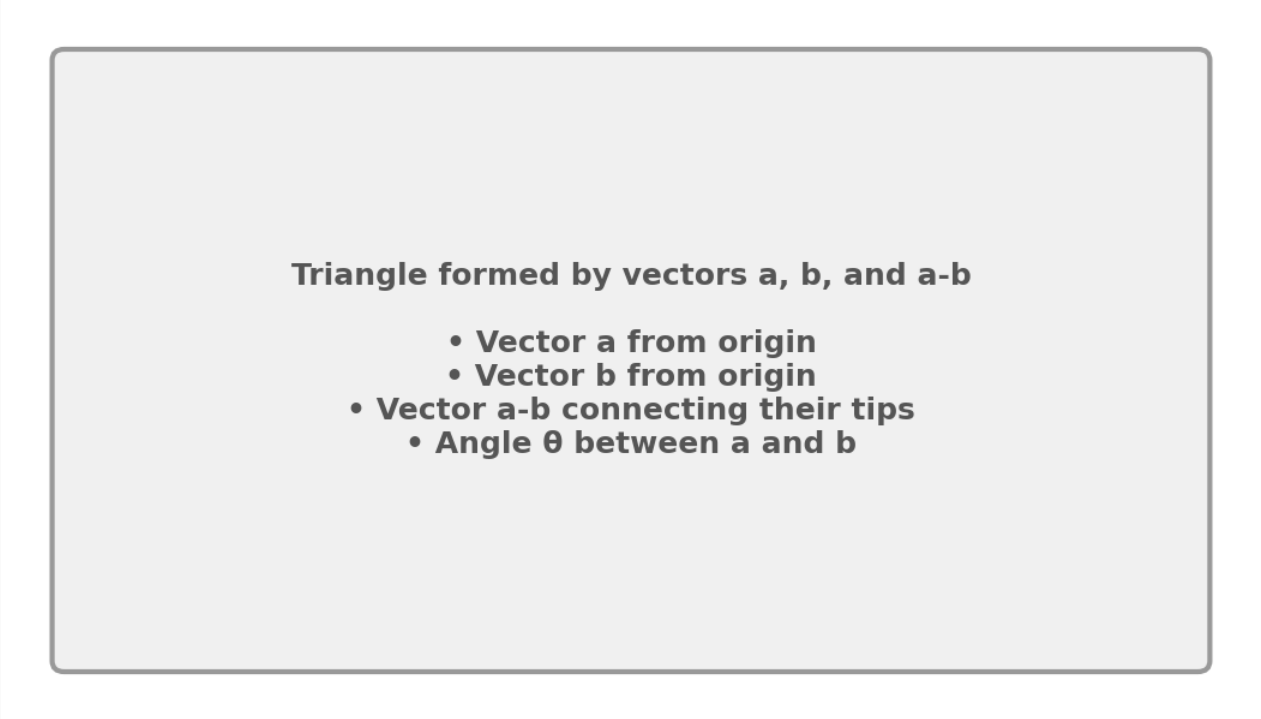
In 3D, if $\mathbf{a} = [a_1, a_2, a_3]$ and $\mathbf{b} = [b_1, b_2, b_3]$:

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + a_3 b_3$$

This is simply: **multiply corresponding components and add them up**.

Let's prove that the algebraic and geometric definitions are equivalent. We'll use the Law of Cosines.

Consider vectors $\mathbf{a}$ and $\mathbf{b}$, and their difference $\mathbf{c} = \mathbf{a} - \mathbf{b}$:



By the Law of Cosines:

$$||\mathbf{c}||^2 = ||\mathbf{a}||^2 + ||\mathbf{b}||^2 - 2||\mathbf{a}|| \cdot ||\mathbf{b}||\cos(\theta)$$

But we also know that:

$$||\mathbf{c}||^2 = ||\mathbf{a} - \mathbf{b}||^2 = (\mathbf{a} - \mathbf{b}) \cdot (\mathbf{a} - \mathbf{b})$$

Expanding using the algebraic definition:

$$(\mathbf{a} - \mathbf{b}) \cdot (\mathbf{a} - \mathbf{b}) = \mathbf{a} \cdot \mathbf{a} - 2\mathbf{a} \cdot \mathbf{b} + \mathbf{b} \cdot \mathbf{b}$$

$$= ||\mathbf{a}||^2 - 2\mathbf{a} \cdot \mathbf{b} + ||\mathbf{b}||^2$$

Comparing the two expressions:

$$||\mathbf{a}||^2 + ||\mathbf{b}||^2 - 2||\mathbf{a}|| \cdot ||\mathbf{b}||\cos(\theta) = ||\mathbf{a}||^2 - 2\mathbf{a} \cdot \mathbf{b} + ||\mathbf{b}||^2$$

Simplifying:

$$\mathbf{a} \cdot \mathbf{b} = ||\mathbf{a}|| \cdot ||\mathbf{b}||\cos(\theta)$$

This proves the equivalence! Let's implement both methods:

```
def dot_product_algebraic(a, b):
    """
    Calculate dot product using algebraic definition.
    a · b = sum(a_i * b_i) for all components i
    """
    if len(a) != len(b):
        raise ValueError("Vectors must have same dimension")

    result = 0
    for i in range(len(a)):
        result += a[i] * b[i]  # Multiply corresponding components and add

    return result

def dot_product_geometric(a, b):
    """
    Calculate dot product using geometric definition.
    a · b = ||a|| * ||b|| * cos(θ)
    Note: This requires knowing the angle, which we typically compute FROM the dot product!
    """
    magnitude_a = np.sqrt(sum(x**2 for x in a))  # ||a|| = sqrt(a1² + a2² + ...)
    magnitude_b = np.sqrt(sum(x**2 for x in b))  # ||b|| = sqrt(b1² + b2² + ...)

    # To get angle, we'd need to use the algebraic dot product!
    # This shows why the algebraic definition is more practical
    algebraic_dot = dot_product_algebraic(a, b)
    cos_theta = algebraic_dot / (magnitude_a * magnitude_b)

    return magnitude_a * magnitude_b * cos_theta

# Test both methods
a = [3, 4]
b = [2, 1]

print(f"Algebraic method: {dot_product_algebraic(a, b)}")
print(f"Geometric method: {dot_product_geometric(a, b)}")
print(f"NumPy's method: {np.dot(a, b)}")
```

Properties of Dot Products

The dot product has several important properties that make it useful:

1. **Commutative**: $\mathbf{a} \cdot \mathbf{b} = \mathbf{b} \cdot \mathbf{a}$
2. **Distributive**: $\mathbf{a} \cdot (\mathbf{b} + \mathbf{c}) = \mathbf{a} \cdot \mathbf{b} + \mathbf{a} \cdot \mathbf{c}$
3. **Scalar multiplication**: $(k\mathbf{a}) \cdot \mathbf{b} = k(\mathbf{a} \cdot \mathbf{b})$

4. **Self dot product**: $\mathbf{a} \cdot \mathbf{a} = ||\mathbf{a}||^2$

Let's verify these properties with code:

```
# Define test vectors
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
c = np.array([7, 8, 9])
k = 2.5  # scalar

# Property 1: Commutative
print("1. Commutative Property:")
print(f"   a · b = {np.dot(a, b)}")
print(f"   b · a = {np.dot(b, a)}")
print(f"   Equal? {np.dot(a, b) == np.dot(b, a)}\n")

# Property 2: Distributive
print("2. Distributive Property:")
left_side = np.dot(a, b + c)
right_side = np.dot(a, b) + np.dot(a, c)
print(f"   a · (b + c) = {left_side}")
print(f"   a · b + a · c = {right_side}")
print(f"   Equal? {np.allclose(left_side, right_side)}\n")

# Property 3: Scalar multiplication
print("3. Scalar Multiplication:")
left_side = np.dot(k * a, b)
right_side = k * np.dot(a, b)
print(f"   (k*a) · b = {left_side}")
print(f"   k*(a · b) = {right_side}")
print(f"   Equal? {np.allclose(left_side, right_side)}\n")

# Property 4: Self dot product
print("4. Self Dot Product:")
self_dot = np.dot(a, a)
magnitude_squared = np.linalg.norm(a)**2
print(f"   a · a = {self_dot}")
print(f"   ||a||² = {magnitude_squared}")
print(f"   Equal? {np.allclose(self_dot, magnitude_squared)}")
```

Orthogonality: When Vectors are Perpendicular

A crucial concept in linear algebra and machine learning is **orthogonality**. Two vectors are orthogonal (perpendicular) if their dot product is zero:

$$\mathbf{a} \perp \mathbf{b} \iff \mathbf{a} \cdot \mathbf{b} = 0$$

Why? From the geometric definition: $\mathbf{a} \cdot \mathbf{b} = ||\mathbf{a}|| \cdot ||\mathbf{b}|| \cos(\theta)$

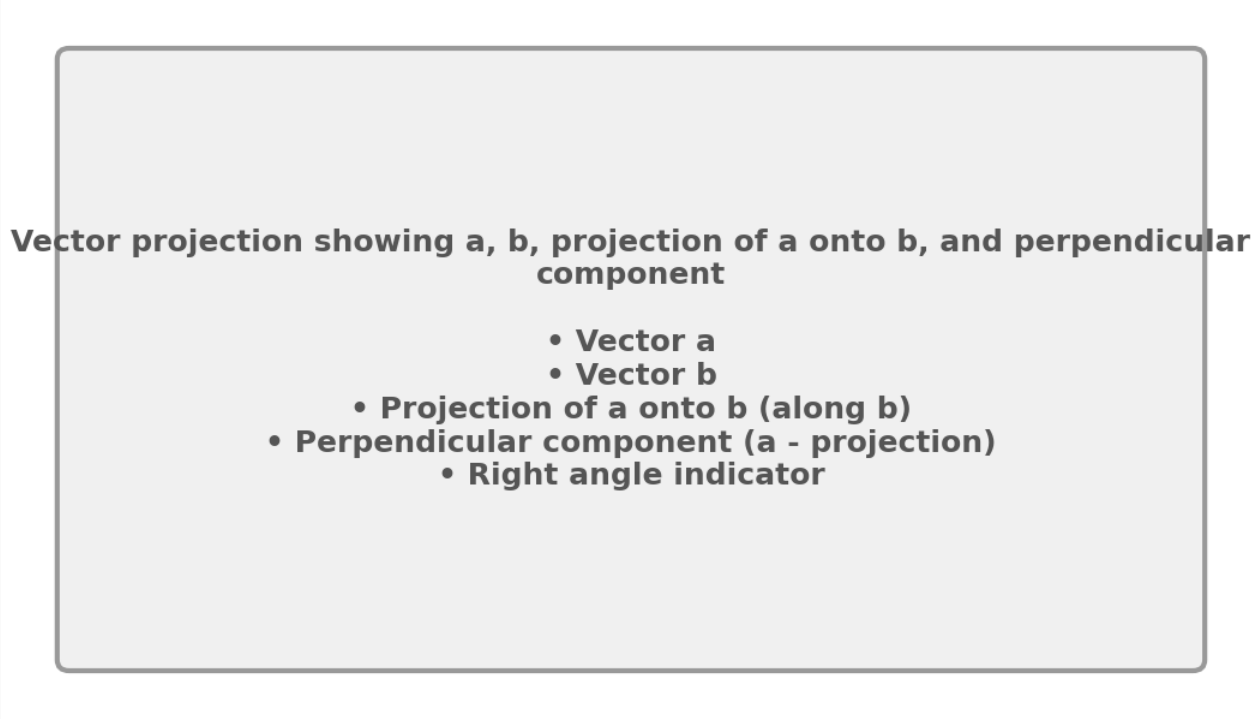
If $\theta = 90°$, then $\cos(90°) = 0$, so the dot product is zero!

```
# Examples of orthogonal vectors
def check_orthogonality(a, b):
    """Check if two vectors are orthogonal (perpendicular)"""
    dot_prod = np.dot(a, b)
    is_orthogonal = np.allclose(dot_prod, 0)  # Use allclose for floating point comparison

    print(f"Vector a: {a}")
    print(f"Vector b: {b}")
    print(f"Dot product: {dot_prod}")
    print(f"Orthogonal? {is_orthogonal}\n")

    return is_orthogonal

# Test cases
print("Checking orthogonality:\n")

# Case 1: Standard basis vectors are orthogonal
check_orthogonality(np.array([1, 0]), np.array([0, 1]))

# Case 2: General orthogonal vectors
check_orthogonality(np.array([2, 3]), np.array([-3, 2]))

# Case 3: Not orthogonal
check_orthogonality(np.array([1, 1]), np.array([1, 2]))

# Visualize orthogonal vectors
plt.figure(figsize=(10, 5))

# Subplot 1: Orthogonal vectors
plt.subplot(1, 2, 1)
v1 = np.array([3, 0])
v2 = np.array([0, 2])
plt.quiver(0, 0, v1[0], v1[1], angles='xy', scale_units='xy', scale=1, color='blue', width=0.01)
plt.quiver(0, 0, v2[0], v2[1], angles='xy', scale_units='xy', scale=1, color='red', width=0.01)
plt.text(v1[0]/2, v1[1]/2 - 0.3, 'v₁', fontsize=12)
plt.text(v2[0]/2 - 0.3, v2[1]/2, 'v₂', fontsize=12)
plt.title(f'Orthogonal Vectors\nv₁ · v₂ = {np.dot(v1, v2)}')
plt.xlim(-1, 4)
plt.ylim(-1, 3)
plt.grid(True, alpha=0.3)
plt.axhline(y=0, color='k', linewidth=0.5)
plt.axvline(x=0, color='k', linewidth=0.5)

# Subplot 2: Non-orthogonal vectors
plt.subplot(1, 2, 2)
v1 = np.array([3, 1])
v2 = np.array([1, 2])
plt.quiver(0, 0, v1[0], v1[1], angles='xy', scale_units='xy', scale=1, color='blue', width=0.01)
```

```
plt.quiver(0, 0, v2[0], v2[1], angles='xy', scale_units='xy', scale=1, color='red', width=0.01)
plt.text(v1[0]/2, v1[1]/2 + 0.2, 'v₁', fontsize=12)
plt.text(v2[0]/2 - 0.3, v2[1]/2, 'v₂', fontsize=12)
plt.title(f'Non-Orthogonal Vectors\nv₁ · v₂ = {np.dot(v1, v2)}')
plt.xlim(-1, 4)
plt.ylim(-1, 3)
plt.grid(True, alpha=0.3)
plt.axhline(y=0, color='k', linewidth=0.5)
plt.axvline(x=0, color='k', linewidth=0.5)

plt.tight_layout()
plt.show()
```

Vector Projections: Breaking Down Vectors

Now we come to one of the most important applications of dot products: **projections**. The projection of vector $\mathbf{a}$ onto vector $\mathbf{b}$ tells us "how much of $\mathbf{a}$ points in the direction of $\mathbf{b}$".



The scalar projection (just the length) is:

$$\text{comp}_{\mathbf{b}}\mathbf{a} = \frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{b}||}$$

The vector projection is:

$$\text{proj}_{\mathbf{b}}\mathbf{a} = \frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{b}||^2}\mathbf{b}$$

Let's derive this step by step:

1. We want to find how much of $\mathbf{a}$ points in the direction of $\mathbf{b}$
2. The unit vector in direction of $\mathbf{b}$ is $\hat{\mathbf{b}} = \frac{\mathbf{b}}{||\mathbf{b}||}$
3. The scalar projection is $\mathbf{a} \cdot \hat{\mathbf{b}} = \mathbf{a} \cdot \frac{\mathbf{b}}{||\mathbf{b}||} = \frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{b}||}$
4. To get the vector projection, multiply by the unit vector: $\frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{b}||} \cdot \frac{\mathbf{b}}{||\mathbf{b}||} = \frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{b}||^2}\mathbf{b}$

```
def vector_projection(a, b):
    """
    Project vector a onto vector b.
    Returns the vector projection of a onto b.
    """
    # Calculate the dot product a · b
    dot_product = np.dot(a, b)

    # Calculate ||b||²
    b_magnitude_squared = np.dot(b, b)

    # Avoid division by zero
    if b_magnitude_squared == 0:
        raise ValueError("Cannot project onto zero vector")

    # Calculate projection: (a · b / ||b||²) * b
    projection = (dot_product / b_magnitude_squared) * b

    return projection

def scalar_projection(a, b):
    """
    Calculate the scalar projection of a onto b.
    This is just the length of the vector projection.
    """
    dot_product = np.dot(a, b)
    b_magnitude = np.linalg.norm(b)

    if b_magnitude == 0:
        raise ValueError("Cannot project onto zero vector")

    return dot_product / b_magnitude

# Example: Project vector a onto vector b
a = np.array([4, 3])
b = np.array([2, 0])  # b points along x-axis

proj = vector_projection(a, b)
scalar_proj = scalar_projection(a, b)

print(f"Vector a: {a}")
print(f"Vector b: {b}")
print(f"Vector projection of a onto b: {proj}")
print(f"Scalar projection of a onto b: {scalar_proj}")

# The perpendicular component
perp = a - proj
print(f"Perpendicular component: {perp}")
```

```
print(f"Check: proj · perp = {np.dot(proj, perp):.6f} (should be 0)")

# Visualize the projection
plt.figure(figsize=(8, 8))

# Draw vectors
plt.quiver(0, 0, a[0], a[1], angles='xy', scale_units='xy', scale=1, color='blue', width=0.01, lab
plt.quiver(0, 0, b[0], b[1], angles='xy', scale_units='xy', scale=1, color='green', width=0.01, la
plt.quiver(0, 0, proj[0], proj[1], angles='xy', scale_units='xy', scale=1, color='red', width=0.01

# Draw the perpendicular line
plt.plot([proj[0], a[0]], [proj[1], a[1]], 'k--', alpha=0.5)

# Draw right angle indicator
right_angle_size = 0.3
plt.plot([proj[0], proj[0]], [proj[1], proj[1] + right_angle_size], 'k-', linewidth=1)
plt.plot([proj[0], proj[0] + right_angle_size], [proj[1] + right_angle_size, proj[1] + right_angle
plt.plot([proj[0] + right_angle_size, proj[0] + right_angle_size], [proj[1] + right_angle_size, pr

plt.xlim(-1, 5)
plt.ylim(-1, 4)
plt.grid(True, alpha=0.3)
plt.axhline(y=0, color='k', linewidth=0.5)
plt.axvline(x=0, color='k', linewidth=0.5)
plt.legend()
plt.title('Vector Projection')
plt.axis('equal')
plt.show()
```

Orthogonal Decomposition

Every vector can be decomposed into components parallel and perpendicular to another vector:

$$\mathbf{a} = \text{proj} < em > \mathbf{ba} + (\mathbf{a} - \text{proj} < /em > \mathbf{ba})$$

This decomposition is fundamental in many ML algorithms: - **Principal Component Analysis (PCA)** projects data onto orthogonal directions - **Gram-Schmidt process** creates orthogonal basis vectors - **QR decomposition** factorizes matrices using orthogonal components

```
def orthogonal_decomposition(a, b):
    """
    Decompose vector a into components parallel and perpendicular to b.
    Returns (parallel_component, perpendicular_component)
    """
    parallel = vector_projection(a, b)
    perpendicular = a - parallel

    return parallel, perpendicular

# Example with multiple vectors
vectors_to_decompose = [
    np.array([3, 4]),
    np.array([1, 5]),
    np.array([-2, 3])
]
reference_vector = np.array([1, 0])  # Decompose relative to x-axis

plt.figure(figsize=(15, 5))

for i, vec in enumerate(vectors_to_decompose):
    parallel, perpendicular = orthogonal_decomposition(vec, reference_vector)

    plt.subplot(1, 3, i+1)

    # Draw original vector
    plt.quiver(0, 0, vec[0], vec[1], angles='xy', scale_units='xy', scale=1,
               color='blue', width=0.01, label=f'v = {vec}')

    # Draw parallel component
    plt.quiver(0, 0, parallel[0], parallel[1], angles='xy', scale_units='xy', scale=1,
               color='red', width=0.01, label=f'parallel = {parallel}')

    # Draw perpendicular component from tip of parallel
    plt.quiver(parallel[0], parallel[1], perpendicular[0], perpendicular[1],
               angles='xy', scale_units='xy', scale=1, color='green', width=0.01,
               label=f'perp = {perpendicular}')

    # Draw reference vector
    plt.quiver(0, 0, reference_vector[0]*3, reference_vector[1]*3,
               angles='xy', scale_units='xy', scale=1, color='gray',
               width=0.005, alpha=0.5, label='reference')

    plt.xlim(-3, 4)
    plt.ylim(-1, 6)
    plt.grid(True, alpha=0.3)
    plt.axhline(y=0, color='k', linewidth=0.5)
    plt.axvline(x=0, color='k', linewidth=0.5)
```

```
    plt.legend(fontsize=8)
    plt.title(f'Decomposition {i+1}')
    plt.axis('equal')

plt.tight_layout()
plt.show()
```

Applications in Machine Learning

Let's explore how dot products and projections are used in real ML contexts:

1. Cosine Similarity

Cosine similarity measures the angle between vectors, ignoring their magnitudes:

$$\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{a}|| \cdot ||\mathbf{b}||}$$

This is widely used in: - Text similarity (comparing word embeddings) - Recommendation systems - Image retrieval

```
def cosine_similarity(a, b):
    """
    Calculate cosine similarity between two vectors.
    Returns value between -1 and 1:
    - 1: vectors point in same direction
    - 0: vectors are orthogonal
    - -1: vectors point in opposite directions
    """
    dot_product = np.dot(a, b)
    magnitude_a = np.linalg.norm(a)
    magnitude_b = np.linalg.norm(b)

    if magnitude_a == 0 or magnitude_b == 0:
        raise ValueError("Cannot compute cosine similarity with zero vector")

    return dot_product / (magnitude_a * magnitude_b)

# Example: Document similarity using word counts
# Imagine each dimension represents a word's frequency
doc1 = np.array([3, 1, 0, 2, 0])  # "machine learning is fun"
doc2 = np.array([2, 1, 0, 3, 0])  # "machine learning is great"
doc3 = np.array([0, 0, 4, 0, 2])  # "python programming tutorial"

print("Document Similarity Analysis:")
print(f"Similarity(doc1, doc2) = {cosine_similarity(doc1, doc2):.3f}")  # High - similar content
print(f"Similarity(doc1, doc3) = {cosine_similarity(doc1, doc3):.3f}")  # Low - different topics
print(f"Similarity(doc2, doc3) = {cosine_similarity(doc2, doc3):.3f}")  # Low - different topics

# Visualize in 2D (using first two dimensions for simplicity)
plt.figure(figsize=(8, 8))

docs_2d = [doc[:2] for doc in [doc1, doc2, doc3]]
colors = ['blue', 'green', 'red']
labels = ['Doc1: ML is fun', 'Doc2: ML is great', 'Doc3: Python tutorial']

for i, (doc, color, label) in enumerate(zip(docs_2d, colors, labels)):
    plt.quiver(0, 0, doc[0], doc[1], angles='xy', scale_units='xy', scale=1,
               color=color, width=0.01, label=label)

# Draw angle arcs between doc1 and doc2
theta = np.arccos(cosine_similarity(docs_2d[0], docs_2d[1]))
arc = np.linspace(0, theta, 50)
arc_radius = 1.5
plt.plot(arc_radius * np.cos(arc), arc_radius * np.sin(arc), 'k--', alpha=0.5)

plt.xlim(-1, 4)
plt.ylim(-1, 2)
plt.grid(True, alpha=0.3)
```

```
plt.axhline(y=0, color='k', linewidth=0.5)
plt.axvline(x=0, color='k', linewidth=0.5)
plt.legend()
plt.title('Document Vectors (First 2 Dimensions)')
plt.show()
```

2. Linear Regression as Projection

In linear regression, we find the best line by projecting data onto a subspace:

```
# Simple 1D linear regression as projection
np.random.seed(42)
n_points = 50

# Generate data: y = 2x + 1 + noise
x = np.random.uniform(0, 10, n_points)
true_slope = 2
true_intercept = 1
noise = np.random.normal(0, 1, n_points)
y = true_slope * x + true_intercept + noise

# Form design matrix (add column of ones for intercept)
X = np.column_stack([np.ones(n_points), x])  # Shape: (n_points, 2)

# The optimal weights are the projection of y onto column space of X
# w = (X^T X)^(-1) X^T y
XtX = np.dot(X.T, X)  # X transpose times X
Xty = np.dot(X.T, y)  # X transpose times y
w_optimal = np.linalg.solve(XtX, Xty)  # Solve instead of inverse for numerical stability

print(f"Optimal weights: intercept = {w_optimal[0]:.3f}, slope = {w_optimal[1]:.3f}")
print(f"True values: intercept = {true_intercept}, slope = {true_slope}")

# Predictions are the projection of y onto column space of X
y_pred = np.dot(X, w_optimal)

# Visualize
plt.figure(figsize=(10, 6))
plt.scatter(x, y, alpha=0.6, label='Data points')
plt.plot(x, y_pred, 'r-', linewidth=2, label=f'Fitted line: y = {w_optimal[0]:.2f} + {w_optimal[1]
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.3)
plt.title('Linear Regression as Projection')
plt.show()

# Show residuals are orthogonal to predictions
residuals = y - y_pred
print(f"\nDot product of residuals and predictions: {np.dot(residuals, y_pred):.6f}")
print("(Should be close to 0 - residuals are orthogonal to projection)")
```

Worked Examples

Example 1: Finding the Angle Between Vectors

Problem: Given vectors $\mathbf{u} = [3, 4]$ and $\mathbf{v} = [1, 2]$, find the angle between them.

Solution:

Step 1: Calculate the dot product using the algebraic definition

$$\mathbf{u} \cdot \mathbf{v} = (3)(1) + (4)(2) = 3 + 8 = 11$$

Step 2: Calculate the magnitudes

$$||\mathbf{u}|| = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

$$||\mathbf{v}|| = \sqrt{1^2 + 2^2} = \sqrt{1 + 4} = \sqrt{5}$$

Step 3: Use the geometric formula to find the angle

$$\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{||\mathbf{u}|| \cdot ||\mathbf{v}||} = \frac{11}{5\sqrt{5}} = \frac{11}{5\sqrt{5}} \cdot \frac{\sqrt{5}}{\sqrt{5}} = \frac{11\sqrt{5}}{25}$$

Step 4: Calculate the angle

$$\theta = \arccos\left(\frac{11\sqrt{5}}{25}\right)$$

Let's verify with code:

```
# Define vectors
u = np.array([3, 4])
v = np.array([1, 2])

# Step 1: Dot product
dot_prod = np.dot(u, v)
print(f"Step 1: u · v = {dot_prod}")

# Step 2: Magnitudes
mag_u = np.linalg.norm(u)
mag_v = np.linalg.norm(v)
print(f"Step 2: ||u|| = {mag_u}, ||v|| = {mag_v:.3f}")

# Step 3: Cosine of angle
cos_theta = dot_prod / (mag_u * mag_v)
print(f"Step 3: cos(θ) = {cos_theta:.4f}")

# Step 4: Angle
theta_rad = np.arccos(cos_theta)
theta_deg = np.degrees(theta_rad)
print(f"Step 4: θ = {theta_rad:.4f} radians = {theta_deg:.2f} degrees")

# Verify calculation
print(f"\nVerification: {mag_u} * {mag_v:.3f} * {cos_theta:.4f} = {mag_u * mag_v * cos_theta:.1f}
```

Example 2: Projecting Force onto an Incline

Problem: A box on a 30° incline experiences a gravitational force of $\mathbf{F} = [0, -10]$ N (pointing straight down). Find: 1. The component of force along the incline 2. The component perpendicular to the incline

Solution:

The incline direction vector at 30° is $\mathbf{d} = [\cos(30°), -\sin(30°)] = [\frac{\sqrt{3}}{2}, -\frac{1}{2}]$

Step 1: Calculate the projection of force onto incline direction

$$\text{proj}_{\mathbf{d}}\mathbf{F} = \frac{\mathbf{F} \cdot \mathbf{d}}{||\mathbf{d}||^2}\mathbf{d}$$

First, find $\mathbf{F} \cdot \mathbf{d}$:

$$\mathbf{F} \cdot \mathbf{d} = (0)\left(\frac{\sqrt{3}}{2}\right) + (-10)\left(-\frac{1}{2}\right) = 0 + 5 = 5$$

Then, find $||\mathbf{d}||^2$:

$$||\mathbf{d}||^2 = \left(\frac{\sqrt{3}}{2}\right)^2 + \left(-\frac{1}{2}\right)^2 = \frac{3}{4} + \frac{1}{4} = 1$$

So:

$$\text{proj}_{\mathbf{d}}\mathbf{F} = \frac{5}{1}\mathbf{d} = 5\left[\frac{\sqrt{3}}{2},\ -\frac{1}{2}\right] = \left[\frac{5\sqrt{3}}{2},\ -\frac{5}{2}\right]$$

Step 2: The perpendicular component is:

$$\mathbf{F} < em > \perp = \mathbf{F} - \text{proj} < /em > \mathbf{dF} = [0,\ -10] - \left[\frac{5\sqrt{3}}{2},\ -\frac{5}{2}\right] = \left[-\frac{5\sqrt{3}}{2},\ -\frac{15}{2}\right]$$

```
# Problem setup
angle = 30  # degrees
F = np.array([0, -10])  # Gravitational force pointing down

# Incline direction vector
angle_rad = np.radians(angle)
d = np.array([np.cos(angle_rad), -np.sin(angle_rad)])

print(f"Force vector: F = {F}")
print(f"Incline direction: d = [{d[0]:.3f}, {d[1]:.3f}]")

# Calculate projection
F_parallel = vector_projection(F, d)
F_perpendicular = F - F_parallel

print(f"\nComponent along incline: {F_parallel}")
print(f"Magnitude along incline: {np.linalg.norm(F_parallel):.3f} N")
print(f"\nComponent perpendicular to incline: {F_perpendicular}")
print(f"Magnitude perpendicular: {np.linalg.norm(F_perpendicular):.3f} N")

# Visualize
plt.figure(figsize=(10, 8))

# Draw incline
incline_length = 12
incline_end = incline_length * d
plt.plot([0, incline_end[0]], [0, incline_end[1]], 'k-', linewidth=3, label='Incline')

# Draw force vectors
origin = np.array([5, -2.5])  # Position box on incline
scale = 0.5  # Scale down for visibility

plt.quiver(origin[0], origin[1], scale*F[0], scale*F[1],
           angles='xy', scale_units='xy', scale=1, color='blue', width=0.02, label='Gravity (F)')
plt.quiver(origin[0], origin[1], scale*F_parallel[0], scale*F_parallel[1],
           angles='xy', scale_units='xy', scale=1, color='red', width=0.02, label='F parallel')
plt.quiver(origin[0] + scale*F_parallel[0], origin[1] + scale*F_parallel[1],
           scale*F_perpendicular[0], scale*F_perpendicular[1],
           angles='xy', scale_units='xy', scale=1, color='green', width=0.02, label='F perpendicul

# Draw box
box_size = 0.8
box = plt.Rectangle((origin[0] - box_size/2, origin[1] - box_size/2),
                    box_size, box_size, fill=True, color='brown', alpha=0.5)
plt.gca().add_patch(box)

# Labels and formatting
plt.xlim(-2, 12)
```

```
plt.ylim(-8, 2)
plt.grid(True, alpha=0.3)
plt.axhline(y=0, color='k', linewidth=0.5)
plt.axvline(x=0, color='k', linewidth=0.5)
plt.legend()
plt.title(f'Force Decomposition on {angle}° Incline')
plt.axis('equal')
plt.show()

# Verify orthogonality
print(f"\nVerification - dot product of components: {np.dot(F_parallel, F_perpendicular):.6f}")
print("(Should be 0 - components are orthogonal)")
```

Example 3: Finding the Closest Point on a Line

Problem: Find the point on the line passing through origin in direction $\mathbf{v} = [2, 1]$ that is closest to point $P = (5, 1)$.

Solution: The closest point is the projection of $\vec{OP}$ onto the line direction $\mathbf{v}$.

```
# Define the line direction and the point
v = np.array([2, 1])  # Line direction
P = np.array([5, 1])  # External point

# The closest point on the line is the projection of P onto v
closest_point = vector_projection(P, v)

# Distance from P to the line
distance_vector = P - closest_point
distance = np.linalg.norm(distance_vector)

print(f"Line direction: v = {v}")
print(f"External point: P = {P}")
print(f"Closest point on line: {closest_point}")
print(f"Distance from P to line: {distance:.3f}")

# Visualize
plt.figure(figsize=(10, 8))

# Draw the line (extended in both directions)
t = np.linspace(-2, 4, 100)
line_x = t * v[0]
line_y = t * v[1]
plt.plot(line_x, line_y, 'b-', linewidth=2, label='Line through origin')

# Plot points
plt.plot(P[0], P[1], 'ro', markersize=10, label=f'Point P = {P}')
plt.plot(closest_point[0], closest_point[1], 'go', markersize=10,
         label=f'Closest point = ({closest_point[0]:.1f}, {closest_point[1]:.1f})')
plt.plot(0, 0, 'ko', markersize=8, label='Origin')

# Draw distance line
plt.plot([P[0], closest_point[0]], [P[1], closest_point[1]], 'r--', linewidth=2,
         label=f'Distance = {distance:.2f}')

# Draw vectors
plt.quiver(0, 0, P[0], P[1], angles='xy', scale_units='xy', scale=1,
           color='gray', width=0.005, alpha=0.5)
plt.quiver(0, 0, closest_point[0], closest_point[1], angles='xy', scale_units='xy',
           scale=1, color='green', width=0.005, alpha=0.5)

# Draw right angle indicator
right_angle_size = 0.3
perp_direction = distance_vector / distance  # Unit vector in perpendicular direction
corner = closest_point + right_angle_size * perp_direction
line_direction = v / np.linalg.norm(v)
plt.plot([corner[0], corner[0] + right_angle_size * line_direction[0]],
         [corner[1], corner[1] + right_angle_size * line_direction[1]], 'k-', linewidth=1)
```

```
plt.plot([corner[0], closest_point[0] + right_angle_size * line_direction[0]],
         [corner[1], closest_point[1] + right_angle_size * line_direction[1]], 'k-', linewidth=1)

plt.xlim(-3, 6)
plt.ylim(-2, 4)
plt.grid(True, alpha=0.3)
plt.axhline(y=0, color='k', linewidth=0.5)
plt.axvline(x=0, color='k', linewidth=0.5)
plt.legend()
plt.title('Finding Closest Point on a Line')
plt.axis('equal')
plt.show()

# Verify the perpendicularity
print(f"\nVerification - angle between distance and line direction:")
print(f"Dot product: {np.dot(distance_vector, v):.6f} (should be 0)")
```

Common Pitfalls

As you work with dot products and projections, watch out for these common mistakes:

1. Confusing Dot Product with Element-wise Multiplication

```
# WRONG: Element-wise multiplication
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
wrong_result = a * b  # This gives [4, 10, 18]

# CORRECT: Dot product
correct_result = np.dot(a, b)  # This gives 32

print(f"Element-wise multiplication: {wrong_result}")
print(f"Dot product: {correct_result}")
```

2. Forgetting that Dot Product Returns a Scalar

The dot product always returns a single number (scalar), not a vector:

```
# The result is a scalar, not a vector!
v1 = np.array([1, 2])
v2 = np.array([3, 4])
result = np.dot(v1, v2)  # Returns 11, not a vector

print(f"Type of dot product result: {type(result)}")
print(f"Result: {result}")
```

3. Projecting onto Zero Vector

You cannot project onto a zero vector:

```
def safe_projection(a, b):
    """Safely project a onto b with zero-check"""
    if np.allclose(b, 0):
        print("Warning: Cannot project onto zero vector!")
        return np.zeros_like(a)
    return vector_projection(a, b)

# Example
a = np.array([1, 2])
b = np.array([0, 0])  # Zero vector!
result = safe_projection(a, b)
```

4. Confusion Between Scalar and Vector Projection

Remember: - **Scalar projection** = a number (the length of the projection) - **Vector projection** = a vector (points in direction of reference vector)

```
a = np.array([3, 4])
b = np.array([1, 0])

scalar_proj = scalar_projection(a, b)    # Returns 3 (just the x-component)
vector_proj = vector_projection(a, b)    # Returns [3, 0] (a vector)

print(f"Scalar projection (number): {scalar_proj}")
print(f"Vector projection (vector): {vector_proj}")
```

5. Sign of Dot Product

The dot product can be negative! This happens when the angle between vectors is greater than 90°:

```
# Vectors pointing in generally opposite directions
v1 = np.array([1, 0])
v2 = np.array([-1, 0.1])

dot_prod = np.dot(v1, v2)
angle = np.degrees(np.arccos(np.clip(dot_prod / (np.linalg.norm(v1) * np.linalg.norm(v2)), -1, 1))

print(f"Dot product: {dot_prod}")  # Negative!
print(f"Angle: {angle:.1f}°")     # Greater than 90°
```

Summary & Key Takeaways

Today we explored the fundamental operations that connect vectors: **dot products** and **projections**. Here are the key concepts to remember:

Dot Product Basics

- **Geometric definition**: $\mathbf{a} \cdot \mathbf{b} = ||\mathbf{a}|| \cdot ||\mathbf{b}|| \cdot \cos(\theta)$
- **Algebraic definition**: $\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{n} a_i b_i$
- Returns a **scalar** (single number), not a vector
- Measures "how much two vectors point in the same direction"

Key Properties

- **Commutative**: $\mathbf{a} \cdot \mathbf{b} = \mathbf{b} \cdot \mathbf{a}$
- **Distributive**: $\mathbf{a} \cdot (\mathbf{b} + \mathbf{c}) = \mathbf{a} \cdot \mathbf{b} + \mathbf{a} \cdot \mathbf{c}$
- **Self dot product**: $\mathbf{a} \cdot \mathbf{a} = ||\mathbf{a}||^2$

Orthogonality

- Vectors are **orthogonal** (perpendicular) when $\mathbf{a} \cdot \mathbf{b} = 0$
- Orthogonal vectors are independent — knowing one tells you nothing about the other

Projections

- **Scalar projection**: $\text{comp}_{\mathbf{b}}\mathbf{a} = \frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{b}||}$
- **Vector projection**: $\text{proj}_{\mathbf{b}}\mathbf{a} = \frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{b}||^2}\mathbf{b}$
- Every vector can be decomposed into parallel and perpendicular components

ML Applications

- **Cosine similarity**: $\cos(\theta) = \frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{a}|| \cdot ||\mathbf{b}||}$
- **Linear regression** finds projections onto column spaces
- **Neural networks** compute weighted sums (dot products) at every layer

- **Dimensionality reduction** projects data onto lower-dimensional subspaces

Quick Reference Formulas

Operation	Formula	Python Code
Dot Product	$\mathbf{a} \cdot \mathbf{b} = \sum a_i b_i$	`np.dot(a, b)`
Magnitude	$\|\|\mathbf{a}\|\| = \sqrt{\mathbf{a} \cdot \mathbf{a}}$	`np.linalg.norm(a)`
Angle Between Vectors	$\theta = \arccos\left(\frac{\mathbf{a} \cdot \mathbf{b}}{\|\|\mathbf{a}\|\|\|\|\mathbf{b}\|\|}\right)$	`np.arccos(np.dot(a,b)/(norm(a)*norm(b)))`
Vector Projection	$\text{proj}_{\mathbf{b}}\mathbf{a} = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\|\mathbf{b}\|\|^2}\mathbf{b}$	`(np.dot(a,b)/np.dot(b,b)) * b`

Tomorrow's Preview

In our next episode, we'll explore **Matrices and Matrix Multiplication**. We'll see how matrices: - Transform vectors through multiplication - Represent systems of linear equations - Encode geometric transformations (rotation, scaling, shearing) - Form the backbone of neural network layers

Think about these questions for tomorrow: 1. If a dot product combines two vectors into a scalar, what does a matrix do to a vector? 2. How can we represent multiple dot products efficiently? 3. What does it mean to "multiply" two tables of numbers?

Matrices are where linear algebra really comes alive in machine learning — they're how we process entire datasets at once and how neural networks transform information layer by layer. See you tomorrow!

Generated by Archon — AI Learning System